Rapport de Projet
La Course des Pirates

Développement d'Application Orienté Objet

Présenté par : **Victor Chalumeaux**

Enseignant : Mr. Vanderstraeten

Date : 19 mars 2026

Table des matières

1 Architecture et Conception **2**
1.1 Séparation Modèle / Affichage 2
1.2 Respect des contraintes techniques 2

2 Diagramme de Séquence Système **3**

3 Conception Détaillée du Modèle **4**
3.1 Architecture Globale 4
3.2 Structure du package `jeu.pirates` 4
3.3 Structure du package `jeu.plateau` 4
3.4 Structure du package `jeu.journaldebord` 5

4 Focus sur les effets spécifiques **6**
4.1 Activation de la Case Rhum 6

1 Architecture et Conception

L'un des objectifs majeurs de ce projet a été la stricte séparation entre la **logique métier** (le Modèle) et les **interactions utilisateurs** (la Vue).

1.1 Séparation Modèle / Affichage

Le projet est structuré en deux entités distinctes pour garantir une meilleure maintenabilité :

- **Le package** `jeu` : Il contient les règles de mouvement, la gestion du plateau et l'état des pirates.
- **Le package** `journaldebord` : Via la classe `Narrateur`, il gère l'affichage.

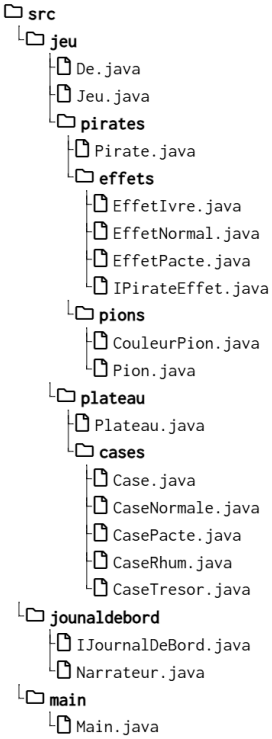

FIGURE 1 – Structure de l'arborescence du projet

1.2 Respect des contraintes techniques

Conformément aux consignes, aucune collection n'a été utilisée. Le code a été audité via l'outil **SonarLint**.

2 Diagramme de Séquence Système

Le diagramme suivant illustre le déroulement global d'une partie.

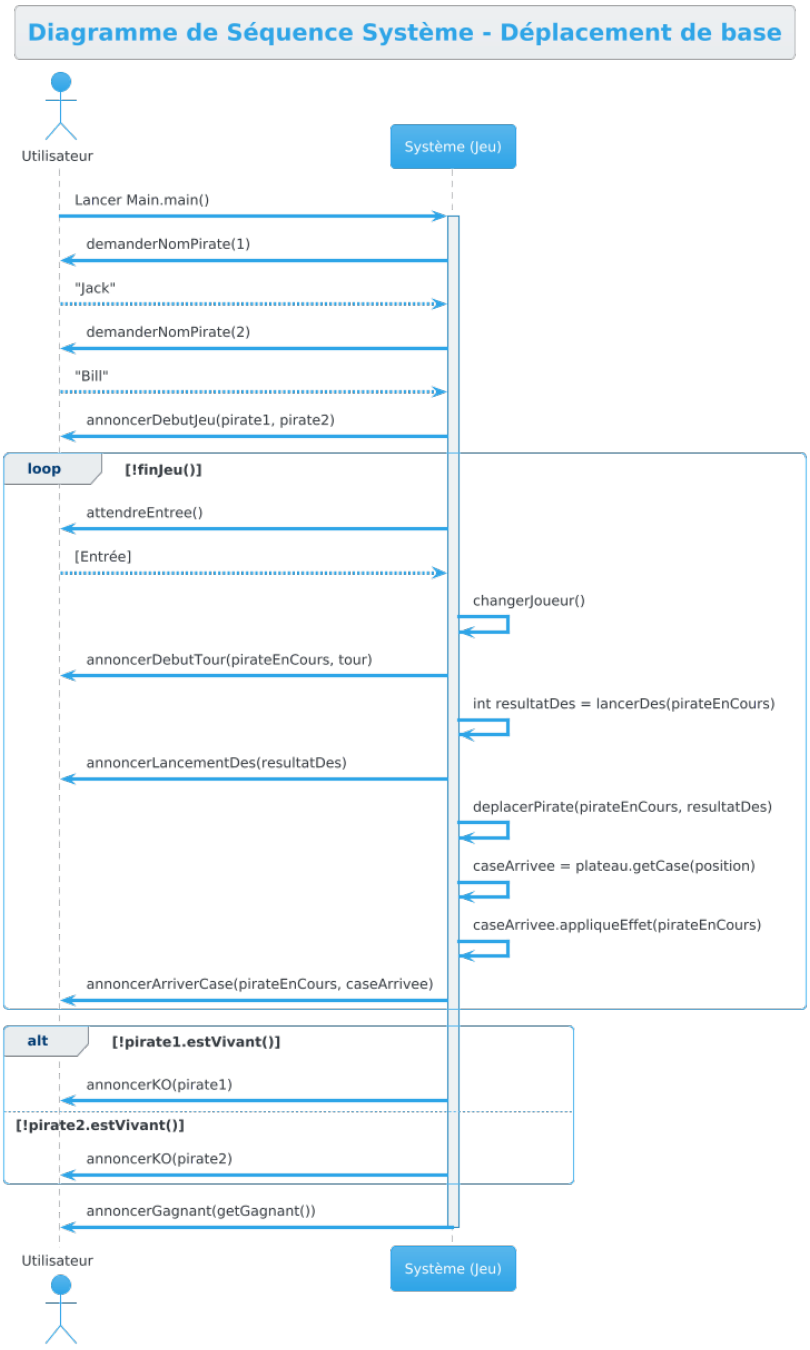

FIGURE 2 – Interaction entre l'utilisateur et le moteur de jeu

Commentaire : L'appel `lancerDes(pirateEnCours)` encapsule toute la complexité du calcul de déplacement.

3 Conception Détaillée du Modèle

3.1 Architecture Globale

Ce diagramme présente les relations de haut niveau entre les composants.

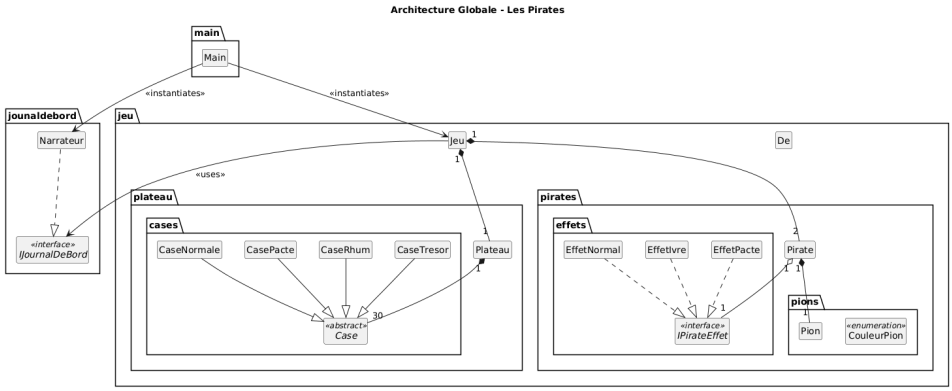

FIGURE 3 – Vue d'ensemble de l'architecture logicielle

3.2 Structure du package `jeu.pirates`

Ce diagramme détaille la gestion des entités et de leurs états. Le `Pirate` délègue la logique de calcul de son mouvement à l'interface `IPirateEffet`, ce qui permet de modifier son comportement dynamiquement sans changer le code de la classe `Pirate`.

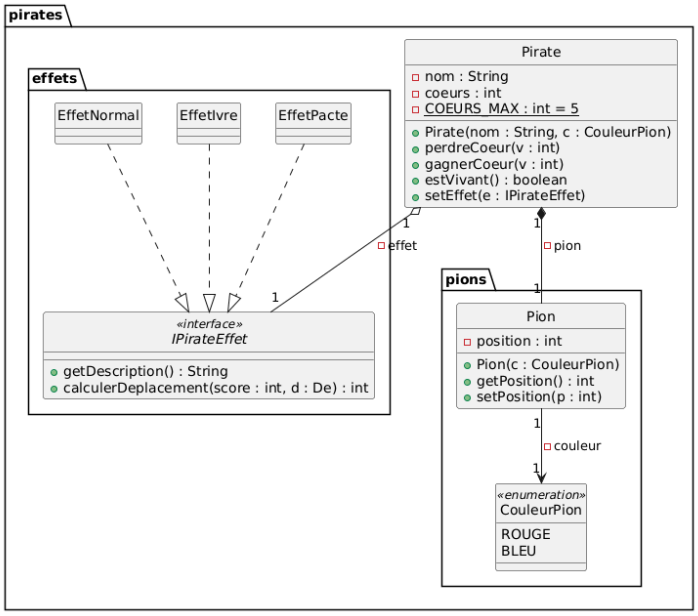

FIGURE 4 – Diagramme des classes liées aux pirates

3.3 Structure du package `jeu.plateau`

Le plateau est composé d'un tableau fixe de 30 cases. L'utilisation d'une classe abstraite `Case` permet d'implémenter le **polymorphisme**.

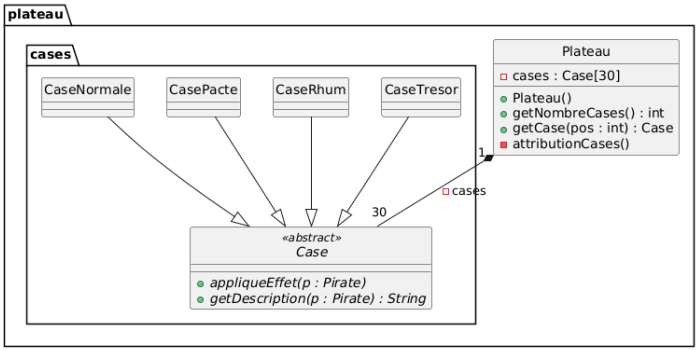

FIGURE 5 – Hiérarchie des cases et structure du plateau

Cette architecture permet de distinguer plusieurs types de comportements lors de l'arrivée d'un pion sur une case :

— **CaseNormale** : Cette case agit comme une zone neutre. Elle réinitialise le comportement du pirate pour que son prochain déplacement soit standard.
— **CaseRhum** : Le pirate récupère 1 point de vie en tombant sur cette case. En contrepartie, il subit un malus d'ébriété qui le forcera à reculer des son résultat lors de son prochain tour de jeu.
— **CasePacte** : Le pirate sacrifie 2 points de vie pour conclure un pacte sombre. Ce sacrifice lui permet de lancer un dé en plus lors de son prochain tour.
— **CaseTresor** : Située à la 30ème et dernière position du plateau, cette case ne possède aucun effet sur les statistiques du pirate. Elle sert de marqueur de victoire pour désigner le gagnant de la partie.

3.4 Structure du package `jeu.journaldebord`

Ce package isole totalement la console du reste du code.

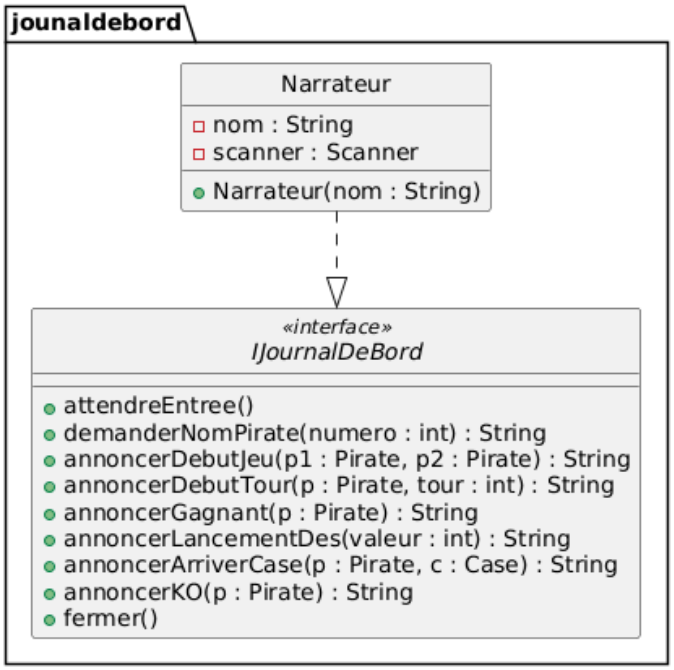

FIGURE 6 – Diagramme des classes du journal de bord

4 Focus sur les effets spécifiques

4.1 Activation de la Case Rhum